

FIT3184 2026 S2 Assignment 1

SmartPark: A Machine Learning-Based Cloud Parking Platform

Due date: 18/Sep/2026 23:55 (Week 8 Friday before the semester break)

1. Synopsis and Background

Parking in the Central Business District (CBD) is often challenging, leading to traffic congestion and driver frustration, as drivers may need to circle to find parking. To address this, a new startup has been formed to develop a smart parking platform that helps users find available car parks and parking lots. The platform will be powered by the You Only Look Once (YOLO) machine-learning vision model. This platform analyses real-time video feeds from car park operators to identify and communicate the most likely available parking spaces to users. As a startup, they don't have much funding, so they decided to hire a full-cycle/full-stack Cloud software engineer (you) to do everything.

This project requires developing a highly scalable, web-based system hosted in a public cloud (GCP, Oracle, Azure) to serve as the backbone of this startup. End users (simulated by automated test scripts) will send HTTP requests to your RESTful web service. The system will process car park images using pre-trained machine learning models and return a structured payload detailing the detected available parking spots, alongside an annotated image if requested. It also provides a simple dashboard that gives the platform operator a high-level overview of the operation.

The web service **must** be developed using **Python's FastAPI** framework. The application will be containerised using Docker and orchestrated using a managed Kubernetes cluster.

2. Project Objectives

This project assignment evaluates competency in cloud-native development, distributed systems, and performance engineering through the following core objectives:

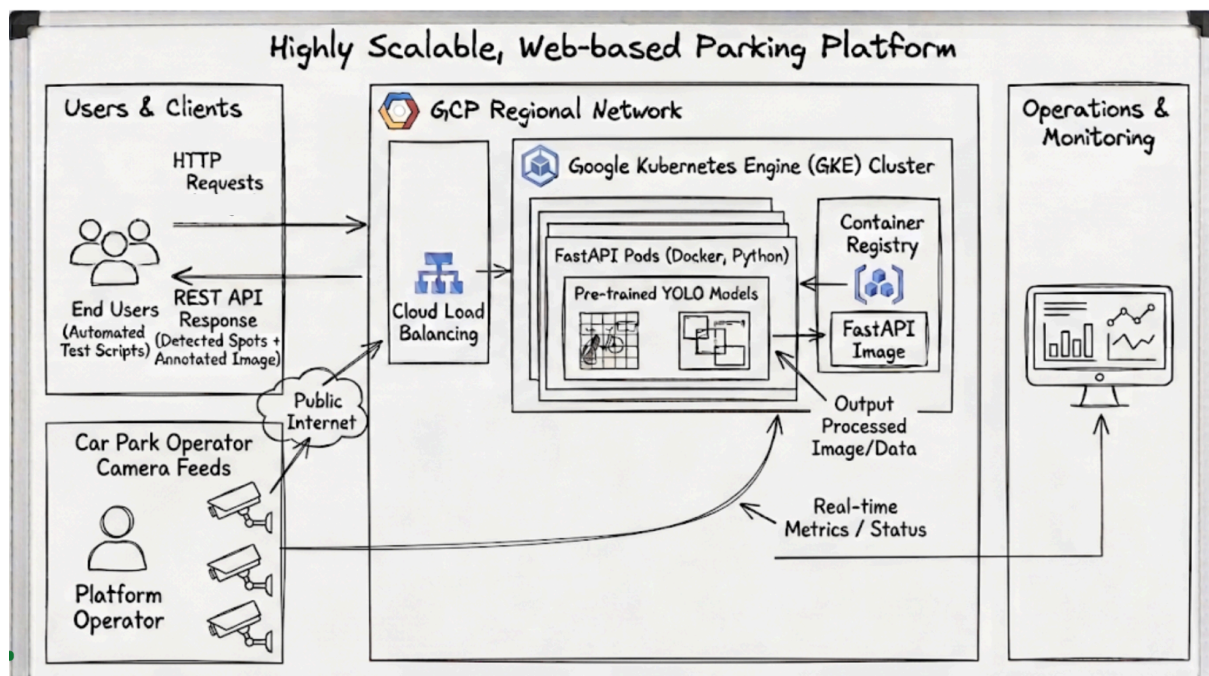
- **API Development:** Constructing a Python-based RESTful API using FastAPI to handle image payloads, perform blocking or asynchronous ML inference safely, and return structured JSON and base64-encoded annotated images. As this is a cloud computing unit, you only need to use the pretrained model; model training or tuning is not required.
- **Containerisation:** Packaging the Python application and system dependencies into an optimised, lightweight Docker image. Making the model loadable and updatable without rebuilding the image.
- **Container Orchestration:** Deploying the containerised application, configuring Kubernetes Deployments and Services (LoadBalancer), and managing strict pod resource requests and limits.

- **Performance Benchmarking:** Using Locust to simulate high-concurrency traffic and measure system latency, throughput (queries per second), and failure rates while keeping resource limitations in mind.
- **Empirical Reporting (500 words):** Analysing system bottlenecks, cost, processing efficiency, and horizontal scaling characteristics in a concise, data-driven benchmark report.

3. The Web Service Architecture

You must develop a RESTful API using the FastAPI framework. The service should be capable of efficiently handling multiple concurrent clients. The web service creates asynchronous task handling for each incoming request, utilising the YOLO engine to detect objects in the image.

After your initial discussion with the startup founder, you drew up and agreed on the following architecture for your first iteration:



Components

- **Users & Clients / End Users:** Simulated by automated scripts, these act as the primary API consumers, submitting car park search requests via **HTTP Requests** and receiving parking availability data (JSON payload) and, if requested, an **Annotated Image** (with bounding boxes) in the **REST API Response**.
- **Car Park Operator Camera:** The continuous source of video stream/images from security cameras installed at parking facilities. To conserve bandwidth, the cameras don't continuously send data to our platform; instead, we send an HTTP GET request to the cameras to request an image of the car park.

- **Platform Operator:** The human administrator who manages the system and requires access to operational dashboards. The platform operator should be able to see car park status and the number of available parking spaces.
- **Google Kubernetes Engine (GKE) Cluster:** The managed Kubernetes environment for automating deployment, scaling, and operational management of application containers.
- **FastAPI Pods (Docker, Python):** The containerised Python web application, built with FastAPI and packaged using Docker, which processes user requests and requests camera images. The visual representation shows multiple replicated pods for handling high traffic.
- **Container Registry:** A private Docker image repository where finalised application images are securely stored and from which GKE pulls them to instantiate pods.
- **Pre-trained YOLO Models:** The real-time object detection logic running *within* the FastAPI application to analyse car park images and identify vehicles and empty spots.
- **Monitoring Dashboard:** Visualises system metrics and status, allowing platform operators to proactively detect and resolve issues.

4. Functionality Requirements

These requirements define the system's core capabilities, focusing on the core business logic of image processing, data flow, and operational management, all within the constraints of the GCP services shown.

Assumptions:

- 1) All car parks are in a small CBD area close to each other; users are willing to go to any of them.
- 2) The camera returns an image for the whole car park.

4.1 Core User-Facing APIs

1. **CORE-API-1:** The system must provide a public REST API endpoint accessible from the Internet to accept HTTP GET requests from End Users (simulated automated test scripts).

Input parameters: num_of_carpark **n** (i.e. 3), uuid: each simulated user has a uuid to associate the request with the response.

Output: A list of the **n** car parks featuring the **greatest count** of available spaces, accompanied by details on parking availability, open spot counts, and confidence scores. To yield **n** results, your application must query at least **2*n random** car parks before returning the top **n** options.

Request: `GET /api/find-carparks?uuid=unique-user-id-12345`

&n=3

Example Output

```
{
  "uuid": "unique-user-id-12345",
  "status": "success",
  "msg": "success", # error message if there is an issue.
  "speed_inference": "xxx ms",
  "requested_n": 3,
  "results": [
    {
      "carpark_id": "CBD_001",
      "available_spaces": 142,
      "confidence_score": 0.985
    },
    {
      "carpark_id": "CBD_042",
      "name": "Market Street East",
      "available_spaces": 98,
      "confidence_score": 0.962
    },
    {
      "carpark_id": "CBD_015",
      "available_spaces": 85,
      "confidence_score": 0.941
    }
  ]
}
```

2. **Core-API-2:** After getting the list of top n car parks, the user may be interested in actually seeing a photo of the car park. Another HTTP GET request will handle this request and return an annotated image.

Request: GET /api/annotate-carpark?carpark_id=CBD_001

Example Output:

```
{
  "carpark_id": "CBD_001",
  "status": "success",
  "msg": "success", # error message if there is an issue.
  "image_base64":
    "iVBORw0KGgoAAAANSUgAAAAEAAAABCAYAAAAfFcSJAAAADUIE
    QVR42mP8z8BQDwAEhQGAhKmMIQAAAABJRU5ErkJggg=="
}
```

4.2 Application Logics & Operational APIs

At startup, the application initialises a configurable number of car parks (10-99). You shall design how the platform discovers the car parks and where to store the configuration/connection details. Each car park exposes an API: **/api/takephoto**. An HTTP GET request to this API returns a base64-encoded image randomly selected from the provided dataset (simulating an image captured by a video camera).

For the operator, please design your own JSON message format and data structure for the following APIs:

1. **OPS-API-1:** Provides a list of all car parks with their IDs and current available parking spots.
2. **OPS-API-2:** Provides the number of users who used the platform in the last 30 seconds. This value should be derived by querying the request logs generated in **OPS-REQ-1**.
3. **OPS-REQ-1:** For operational visibility, you shall implement a mechanism to log the list and/or count of users' requests to the core API.
4. **OPS-REQ-2:** Operational Dashboard (monitoring): Once this application is started, it should provide an operational monitoring page/dashboard to visualise the operation. For this requirement, you can use matplotlib to create a plot on demand or implement a full-fledged dashboard using a frontend framework like Plotly Dash(<https://plotly.com/dash/>).

4.3 Other Requirements

Performance Optimisation

Your application should be optimised to achieve higher concurrency/performance per node/container. For example, repeated requests from the same user can be cached. Also, as a cloud engineer, ask yourself lots of "what-if" questions, even for this simple application. For instance, what if the user sends a large n ($n > 100$)?

Stability and Scalability

The platform must maintain service availability and performance under fluctuating traffic conditions. You must consider load balancing and scaling within the Kubernetes cluster to dynamically adjust replica counts based on CPU utilisation metrics. Furthermore, all pods must be configured with explicit resource requests and limits to ensure predictable resource allocation.

Updateable Model

Machine learning models evolve rapidly through continuous retraining and data updates. Baking large model weights directly into container images causes massive image bloat, sluggish scaling, and inefficient registry storage. This practice tightly couples the rapid data lifecycle of parameters with the slower release cycle of application code. Hence, the application shall keep container images lightweight and mount model weights dynamically at runtime. You shall design a mechanism to implement this feature.

Project Cost Planning and Control

You shall manage cloud costs carefully during the development, deployment, and demo periods. As a guideline, the whole project should cost less than \$20.

Application Logging

The application must implement comprehensive logging and tracing to ensure system observability and facilitate debugging. All logs must capture essential request metadata, including timestamps, severity levels (e.g., DEBUG, INFO, ERROR), service name, and unique request/user UUIDs.

5. Load Generation and Performance Metrics

Using Locust, you will write a Python load-generation script to simulate concurrent user access to your API. The script must continuously send requests to your core API to simulate end-user traffic.

Hint: Because machine learning inference is highly CPU-bound, achieving high concurrency in Python requires careful architectural management. If you simply execute the synchronous YOLO prediction inside a standard asynchronous FastAPI route, you will block the main event loop, causing catastrophic latency spikes for all concurrent requests.

6. Experiments and Benchmark Report

You must run your Locust scripts against your K8S cluster while scaling the Deployment replicas to 1, 2, 4, and 8 to vary the number of active pods. You must submit a concise, data-driven benchmark report (maximum 500 words) containing:

1. **Results Table:** Documenting the maximum users and average response time for the 1, 2, 4, and 8 pod configurations.
2. **Graphical Plots:** Two distinct plots mapping the correlation between concurrent users and average latency across the different scaling configurations.
3. **Performance Analysis:** A technical discussion explaining the scaling efficiency. Identify the primary bottlenecks in your FastAPI/YOLO architecture and justify how to mitigate them.
4. **Cost Analysis:** A detailed analysis of 1) the cloud cost of your project from the beginning to submission; 2) the estimated running cost for a 20-minute demo.

7. Interview/Demo

As part of the assessment process, you will be required to attend a mandatory 20-minute live technical interview with the teaching team to verify the authenticity of your submission and demonstrate your technical understanding. **This is a mandatory requirement;** you shall pay attention to the unit announcement shortly before the submission deadline and book your interview within one week after your submission.

- You must share your screen and demonstrate the live cluster using `kubectl` commands.
- Execute the Locust script in real time to demonstrate application functionality and load balancing.
- You shall adequately explain your own code (regardless of whether GenAI is used for code generation) and demonstrate a good understanding of your cloud infrastructure. **The assessment is based on your competency, regardless of the system's source code or operational status.**

8. Submission

You need to submit 3 files via Moodle:

- A report in PDF format (`firstname_lastname_studentid.pdf`).
- A PDF file containing the Generative AI prompts you have used and their output.
- A `.ZIP` file containing your project artefact, including Dockerfile, source code, YAML files, Locust script, a `readme.md` and any other files you created. The deliverable should be **complete** (except for the pre-trained model), so that anyone who has your project artefact can build and deploy it.

Late submissions and Special Consideration

ALL Special Consideration, including within the semester, is now handled centrally. This means that students **MUST** submit an online Special Consideration form via Monash Connect. For more details, please refer to the **Unit Information** section in Moodle.

There is a **5% penalty per day, including weekends** for a late submission. Also, the cut-off date is 7 days after the due date. No submission will be accepted (i.e. zero mark) after the cut-off date unless you have a special consideration.

9 Additional Information

Where to get help

We have at least two consultation sessions every week; please join if you have any questions. Also, you can ask questions about the assignment in the Assignments section in the Ed Forum accessible on the unit's Moodle Forum page. This is the preferred venue for assignment clarification-type questions. **Students are expected to check the Ed Forum regularly, as responses from the teaching staff are considered official and may constitute amendments or additions to the assignment specification. Clarifications, updates, and additional information will be posted there to help avoid confusion when completing the assignment. Failure to check the forum will not be accepted as a reason for misunderstanding the assignment requirements.**

Mark Release and Review

- Mark will be released within 10 business days after the submission deadline. Please note: for assignments with interviews, this "submission deadline" includes the interview phase, not just the artefact/code submission.
- Reviews and disputes regarding the mark will be accepted for up to 7 days after the release date (including weekends).
- The marking rubric will be released shortly in week 5.

Generative AI Statement

As per the University's [policy](#) on the guidelines and practices pertaining to the usage of Generative AI:

AI & Generative AI tools may be used SELECTIVELY within this assessment.

Where used, AI must be used responsibly, clearly documented, and appropriately acknowledged (see Learn HQ).

Any work submitted for a mark must:

1. Represent a sincere demonstration of your human efforts, skills and subject knowledge that you will be accountable for.
2. Adhere to the guidelines for AI use set for the assessment task.
3. Reflect the University's commitment to academic integrity and ethical behaviour.

Inappropriate AI use and/or AI use without acknowledgement will be considered a breach of academic integrity.

The teaching team encourages students to apply their own critical thinking and reasoning skills when working on assessments, with assistance from GenAI. Generative AI tools may produce inaccurate content, which could negatively impact students' comprehension of the topics.